Information Retrieval and Web Analytics (IRWA) 2026

Part 2:
Indexing and Evaluation

Group members

Name	Student ID
Mauro Paniagua Suetta	u232628
Martí Pascual Barluenga	u232621
Martí Pérez Rodríguez	u232620

GitHub repository

`https://github.com/Martiperez314/IRWA-2026-project`

Tag: `IRWA-2026-part-2`

How to run everything (environment, data files and notebook) is explained in `project_progress/part_2/README.md`.

3 November 2026

1 Introduction

In this part we build a first search engine on top of the corpus we processed in Part 1, and we measure how good it is. The code is split in two reusable files: `myapp/search/algorithms.py` (inverted index, AND search and TF-IDF ranking) and `myapp/search/evaluation.py` (the seven evaluation metrics). All results in this report come from the notebook `project_progress/part_2/part_2.ipynb`, and our own relevance labels are in `our_queries_labels.csv` in the same folder.

The pipeline has five steps, and each one has its own section:

1. Load the processed corpus from Part 1 (28,080 products, field `all_tokens`).
2. Build the inverted index (Section 2).
3. Process the query with `preprocess_text` and run the AND search (Section 2).
4. Rank the matching products with TF-IDF (Section 4).
5. Evaluate the rankings against relevance labels (Sections 5 to 9).

2 Build inverted index (Part 1, point 1)

`create_index` builds a dictionary `{term: {pid: tf}}`: for every term, the products that contain it and how many times. We build it from `all_tokens`, the merged field from Part 1 (title, description, brand, category, sub-category and the useful `product_details` values). It takes 0.6 seconds, has 5,790 terms and covers all 28,080 products. For example, `sweatshirt` appears in 1,417 products.

`conjunctive_search` implements the AND queries: it intersects the posting lists of all the query terms, so a product is returned only if it contains every term.

Decisions.

- **We index `all_tokens`, not only the title.** In Part 1 we saw that 39.7% of the products have no description and that `cotton` (query 1) never appears in the titles of the labelled products, only in the description and the details. With a title only index, query 1 would return almost nothing.
- **The index stores tf, not just the list of products.** A plain list of products is enough for the AND search, but TF-IDF needs to know how many times each term appears in each product. Storing it now means we don't have to scan the corpus again to rank.
- **Queries go through the same `preprocess_text` as the documents.** The index holds stems like `sleev` and `jean`, so the query has to be stemmed the same way or nothing matches.

Query	Terms after processing	Products per term	AND
women full sleeve sweatshirt cotton	women, full, sleev, sweatshirt, cotton	13,699; 7,150; 17,904; 1,417; 19,584	467
men slim jeans blue	men, slim, jean, blue	14,123; 5,646; 2,430; 7,154	219

Table 1: AND search on the two validation queries: each term alone matches thousands of products, the intersection only a few hundred.

Table 1 shows how much the AND rule narrows the results: every term matches thousands of products, but only 467 and 219 products contain all of them.

3 Propose test queries (Part 1, point 2)

We followed the hint and started from the most popular terms, counted by document frequency (notebook 1.2). The top ones are `accessori` (27,189 products), `cloth` (27,121), `wear` (20,925), `cotton` (19,584), `sleev` (17,904), `men` (14,123), `women` (13,699) and `tshirt` (12,899).

A query made only of these words would match thousands of products and wouldn't look like a real search, so we combined them following the pattern people use in a shop: **gender + attribute + product type**. The gender and attribute words are popular (`men`, `women`, `cotton`, `black`, `print`), and the product type is rarer (`kurta`, `polo`, `jacket`, `shoe`, `trouser`), which keeps the AND results to a size we can judge. We also picked a different sub-category for each query, so together they test the engine on different kinds of products (Table 2).

ID	Query	Processed terms	Target sub-category	AND
3	women cotton printed kurta	women, cotton, print, kurta	Kurtas, Ethnic Sets and Bottoms	41
4	men black polo tshirt	men, black, polo, tshirt	Topwear	222
5	women winter hooded jacket	women, winter, hood, jacket	Winter Wear	72
6	men running sports shoes	men, run, sport, shoe	Men's Footwear	74
7	men formal trousers	men, formal, trouser	Bottomwear	237

Table 2: Our five test queries (IDs 1 and 2 are the validation queries).

4 Rank your results: TF-IDF (Part 1, point 3)

The products returned by the AND search are ranked by the cosine similarity between the query vector and the product vector (`rank_tfidf`; `compute_idf` and `compute_doc_norms` are computed once, in 0.2 seconds):

$$w_{t,d} = (1 + \log tf_{t,d}) \cdot \log \frac{N}{df_t} \qquad \text{score}(q,d) = \frac{\sum_{t \in q} w_{t,q}\, w_{t,d}}{\|q\|\,\|d\|}$$

where $N = 28{,}080$ and df_t is the number of products that contain t. `search_in_corpus` puts it all together: process the query, AND search, rank, and return the top k (or all matches).

Decisions.

- **Log-scaled tf.** A word repeated 10 times in a description is not 10 times more relevant than a word that appears once, so we use $1 + \log tf$ instead of the raw count.
- **idf makes rare words count more.** In the validation queries, `sweatshirt` (2.987) and `jean` (2.447) weigh much more than `women` (0.718), `men` (0.687), `sleev` (0.450) or `cotton` (0.360). The product type is what really tells products apart, and idf gives it the highest weight without us having to choose it.
- **Normalising by the vector length.** Without it, products with long descriptions would win just because they contain more words. Dividing by $\|d\|$ also lets a short title only product compete with a long one.

Rank	Score	pid	Title	Price
1	0.755	TSHFDCH4EHXXBRRC	Solid Men Polo Neck Black T-Shirt	1,119
2	0.755	TSHFZBYXXZVG4YG7	Solid Men Polo Neck Black T-Shirt	2,274
3	0.748	TSHEQFQ2FRKWAFZC	Solid Men Polo Neck Black T-Shirt	719

Table 3: Top 3 results for "men black polo tshirt".

Table 3 shows a typical ranking. The top results are exactly what was asked, but the first two have the same score and the same title: the corpus is full of near-duplicates (Part 1), and TF-IDF can't tell them apart. We come back to this in Section 9.

5 Evaluation metrics (Part 2, point 1)

All the metrics are in `evaluation.py`. Each one receives `rels`, the relevance labels of the results in ranking order (for example $[1, 0, 1, \dots]$), and `total_relevant`, the number of relevant products for the query (R below). rel_i is the label at position i. Table 4 lists them, and Table 5 applies them to the validation queries with the conventions below.

Metric	Function	Definition
Precision@K	`precision_at_k`	$\frac{1}{K}\sum_{i=1}^{K} rel_i$
Recall@K	`recall_at_k`	$\frac{1}{R}\sum_{i=1}^{K} rel_i$
Average Precision@K	`avg_precision_at_k`	$\frac{1}{\min(R,K)}\sum_{i=1}^{K} P@i \cdot rel_i$
F1-Score@K	`f1_at_k`	$\frac{2 \cdot P@K \cdot R@K}{P@K + R@K}$
MAP	`mean_average_precision`	mean of AP@K over the queries
MRR	`mean_reciprocal_rank`	mean over the queries of 1/rank of the first relevant result (0 if none in the top K)
NDCG@K	`ndcg_at_k` (uses `dcg_at_k`)	DCG@K $= \sum_{i=1}^{K} \frac{rel_i}{\log_2(i+1)}$, divided by the DCG of the ideal ranking

Table 4: The seven metrics and where they are implemented.

Conventions.

- $K = 10$ for all the results in this report.
- P@K always divides by K, even if fewer than K judged results are returned.
- AP@K divides by $\min(R, K)$, so a query with more than K relevant products can still reach 1 when the whole top K is relevant.
- NDCG uses binary relevance (1 or 0) and the ideal ranking puts all R relevant products first.
- **Only labelled products are judged.** We take the full AND + TF-IDF ranking and keep only the products that have a label, because we can't know if an unlabelled product is relevant. Relevant products that the AND search doesn't return still count in R, so they lower recall.

To check the code we used a small example (notebook 2.1): 10 results $[1, 0, 1, 1, 0, 0, 1, 0, 0, 0]$ with 5 relevant products in total. By hand, P@10 = 4/10 = 0.400, R@10 = 4/5 = 0.800, AP@10 = (1 + 2/3 + 3/4 + 4/7) / 5 = 0.598 and RR = 1, and the functions give exactly these values (plus F1@10 = 0.533 and NDCG@10 = 0.768).

6 Results on the validation queries (Part 2, point 2)

Query	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
1: women full sleeve sweatshirt cotton	1.000	0.769	1.000	0.870	1.000	1.000
2: men slim jeans blue	0.600	0.600	0.569	0.600	1.000	0.716
		MAP@10: 0.785			**MRR@10:** 1.000	

Table 5: Metrics on `validation_labels.csv`.

7 Our relevance judgements (Part 2, point 3a)

We labelled the **top 20 results** of each of our five queries (100 products, `our_queries_labels.csv`): 1 if relevant, 0 if not. Before labelling, we agreed on one general rule and then wrote down how it

applies to each query, so that the labels are consistent.

General rule. A product is relevant only if it matches the gender, the product type and every attribute in the query. Identical products get the same label. Price, rating and stock are ignored (they are not about relevance). When the title and the category disagree, the title decides.

Query	How the rule applies
3: women cotton printed kurta	Women. A ready-to-wear kurta or a set with a kurta; unstitched fabric is 0. Any cotton, including blends (cotton linen, cotton silk). Printed, including polka print; solid, woven or "Self Design" is 0.
4: men black polo tshirt	Men. Polo neck t-shirt. Black is the main colour (solid black, black-white and black mix count; multicolour where black isn't dominant is 0). Any pattern.
5: women winter hooded jacket	Women. A jacket; sweatshirts and pullover hoodies are 0. The listing must say "hooded". "Winter" is rarely written, so we read it as full sleeve: sleeveless jackets, rain jackets and windcheaters are 0.
6: men running sports shoes	Men; women's shoes are 0. Shoes described as running shoes; a generic sports shoe with no mention of running is 0.
7: men formal trousers	Men. Trousers, formal or tailored (a "Formal Trouser" listed under Ethnic Wear still counts). Fit doesn't matter.

Table 6: Our labelling criteria for each test query.

For the shoes we chose the strict option (running must be mentioned), because the general rule asks for every attribute in the query and we didn't want one query to be judged more leniently than the others.

Query	Rel. in top 20	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
3: kurta	18	0.900	0.500	0.707	0.643	0.500	0.780
4: polo	20	1.000	0.500	1.000	0.667	1.000	1.000
5: jacket	14	0.700	0.500	0.700	0.583	1.000	0.801
6: shoes	12	0.800	0.667	0.659	0.727	1.000	0.812
7: trousers	20	1.000	0.500	1.000	0.667	1.000	1.000
		MAP@10: 0.813				**MRR@10:** 0.900	

Table 7: Metrics on our five test queries, with our own labels.

Table 7 shows the results. Most of the 0 labels come from three kinds of products: the unstitched kurta fabric ranked 1st for query 3, hooded sweatshirts (and one jacket with no hood) in query 5, and women's running shoes in query 6 (8 of the top 20).

8 Comments on the evaluation metrics (Part 2, point 3b)

Order inside the top K. P@K, R@K and F1@K only count *how many* relevant results are in the top K; moving them around inside the top 10 doesn't change them. AP, RR and NDCG reward relevant results near the top. Query 3 shows the difference well: P@10 is 0.900, so the top 10 looks almost perfect, but the first result is the unstitched fabric. RR drops to 0.500, AP to 0.707 and NDCG to 0.780. For a user who looks at the first result, this matters a lot, and only the rank-aware metrics see it.

Only the first result: RR and MRR. RR only looks at the position of the first relevant result. It is the right metric when one good answer is enough, but it says nothing about the rest of the list.

MRR is 1.000 on the validation queries, even though query 2 has 4 non-relevant products in its top 10. On our queries MRR is 0.900, and the only thing pulling it down is query 3's first result.

Recall@10 is capped. When a query has more relevant products than K, R@K can't reach 1: queries 3, 4, 5 and 7 have 14 to 20 relevant products, so even a perfect top 10 gives R@10 = 0.500. F1 inherits the cap: queries 4 and 7 have a perfect top 10 but F1@10 = 0.667. Query 6 has a *higher* recall (0.667) than query 4 only because it has fewer relevant products (12), not because it is ranked better. So recall and F1 can't be compared across queries with different R.

The judged pool. We only labelled the top 20 results, so R is the number of relevant products *in that pool*, not in the whole corpus (query 7 has 237 AND results and probably many more relevant trousers). Our recall is a recall relative to the pool, and it is optimistic: products the AND search misses never get judged, so they can't lower it. The validation labels are different: they include relevant products that our search doesn't return (one for query 1, four for query 2), so they can lower recall.

P@10 vs NDCG. NDCG weighs each relevant result by its position. In query 5, the 7 relevant results are positions 1 to 7 and the three sweatshirts are at 8, 9 and 10, so NDCG (0.801) is higher than P@10 (0.700): the mistakes are at the bottom, where the discount is strongest. Query 2 shows the same pattern (0.716 vs 0.600), with its first result relevant (RR = 1.000).

AP vs MAP. AP describes one query; MAP averages it over all queries, so we get one number for the whole system that we can use to compare versions of the engine. The price is that it hides the spread: our MAP of 0.813 mixes two perfect queries (AP = 1.000) with query 6 (0.659).

Validation (MAP 0.785) vs our queries (MAP 0.813). Our queries score a bit higher, for two reasons. First, the pool: our labels only cover what the engine returned, while the validation labels also include relevant products the AND search misses (Section 9). Second, query 2 is hard: its labels include women's jeans as relevant for "men slim jeans blue", which no word-based search would rank for that query (we saw this in Part 1). Queries 4 and 7 are the opposite case: all 20 judged products are relevant, so their metrics are perfect and tell us little about ranking quality, only that the top 20 is clean.

9 Problems and proposed solutions (Part 2, point 3c)

The evidence comes from notebook 2.3.c. Table 8 summarises it, and the paragraphs below explain each problem and our proposed fix.

AND is too strict. One missing word is enough to lose a relevant product: a "Tapered Fit Men Blue Jeans" disappears because it doesn't say `slim`, and a relevant sweatshirt because `cotton` isn't in its text. The more words a query has, the worse this gets. *Fix:* relax the matching. For example, require most of the terms (say, all but one) instead of all of them, or use OR to retrieve and let the ranking push products with more matching terms to the top.

AND vs OR. OR alone isn't the answer either: it returns from 15,033 to 25,503 products, basically the whole catalogue, because words like `men` or `cotton` are everywhere. *Fix:* a middle ground. Require the rare, important terms (the product type, which has a high idf) and treat the common ones (gender, `cotton`) as optional terms that only add to the score.

Problem	Evidence	Aspect
AND is too strict	Query 2: 4 of 10 relevant products not retrieved (3 lack `men`, 1 lacks `slim`); query 1: 1 of 13 lacks `cotton`	Retrieval accuracy
AND and OR are two extremes	AND gives from 41 to 467 results, OR from 15,033 to 25,503	Query formulation
Gender words	"Running Shoes For Women" ranked 3rd for "men running sports shoes"; 8 of the top 20 are women's shoes	Query formulation, field types
Duplicates	Distinct titles in the top 10: 1 for query 2, 2 for queries 1 and 4	Ranking quality
All fields weigh the same	`cotton` is never in the labelled titles; a match in the description counts like one in the title	Field types, indexing
Rating, stock and price ignored	From 0 to 2 out-of-stock products in every top 10 (query 1: ranks 1 and 5); top-10 rating below the average of all results (query 1: 3.41 vs 3.80)	Ranking quality
Wrong product type ranks high	Query 3: unstitched fabric 1st; query 5: sweatshirts at 8, 9 and 10	Ranking quality, field types
No synonyms	`jeans` vs `denim`, `sneakers` vs `shoes`	Indexing, query formulation

Table 8: Main problems of the current search system and the evidence for each.

Gender words are just words. The word `men` appears somewhere in the text of women's running shoes (otherwise the AND search wouldn't return them), and TF-IDF gives it a low weight (0.687) because it is so common. So for "men running sports shoes" the engine can't tell men's from women's shoes, and 8 of our top 20 are women's. Plain word matching can't fix this, because `men` and `women` are different tokens and the search has no idea they are opposites. *Fix:* recognise gender words in the query and use them as a **filter** on a gender field. The *Ideal For* key of `product_details`, which we already extract in Part 1, gives that field.

Duplicates in the top 10. For query 2, all 10 results have the same title ("Slim Men Blue Jeans"), and queries 1 and 4 only have 2 distinct titles. They are the same product in different sizes or sold by different sellers, which we already found in Part 1 (42.1% of products share title, description, brand and price). The metrics count them as several correct results, but the user sees the same thing ten times. *Fix:* group identical products (same title, brand and description) and show one per group, or penalise a result that is too similar to one already shown.

All fields weigh the same. We index the merged `all_tokens`, so a word in the title counts the same as a word buried in a long description, and the engine can't tell which word of the title is the product type. That's how "Cotton Printed Kurta Fabric (Unstitched)", a short product with three query words in its title, beats real kurtas. *Fix:* use the separate `*_tokens` columns we kept in Part 1 and give each field its own weight (more for the title and the details, less for the description), for example with a BM25F-style weighted score.

Rating, stock and price are ignored. The ranking only uses text. Query 1 has out-of-stock products at ranks 1 and 5, and the average rating of its top 10 (3.41) is lower than the average of all its results (3.80), so better products are hidden further down. *Fix:* combine the text score with the numeric fields we prepared in Part 1: push down or hide out-of-stock products, and add a small bonus for rating (only for products that have one, using the `has_rating` flag).

The wrong product type ranks high. In query 5 the hooded sweatshirts share almost all the words of a hooded jacket (`women`, `hood`, `full`, `sleev`), so they get close scores. The same happens with the kurta fabric in query 3. *Fix:* detect the product type in the query and match it against `sub_category` or the *Type* key of the details, either as a filter or as a strong boost.

No synonyms. The engine only matches the exact (stemmed) word: a query for "denim" or "sneakers" would miss products that say "jeans" or "shoes". *Fix:* query expansion with a small list of synonyms for the fashion domain, or word embeddings to find related terms.

10 Conclusions and assumptions

The engine works well for simple queries: the first result is relevant in 6 of our 7 queries, and the MAP is 0.785 on the validation queries and 0.813 on ours. Its main weaknesses are not the TF-IDF formula but what the ranking ignores: the AND rule loses relevant products for one missing word, gender and product type are treated as normal words, every field weighs the same, and duplicates and out-of-stock products fill the top 10. We also learned that the metrics have to be read together: P@10 hides a bad first result, and recall and F1 are capped by the number of relevant products.

Assumptions (summary).

- The index is built on `all_tokens` from Part 1, and queries use the same `preprocess_text`.
- Queries are conjunctive (AND), as the statement asks; ranking uses log-scaled tf, idf and cosine normalisation.
- $K = 10$ for all metrics; P@K divides by K and AP@K by $\min(R, K)$; relevance is binary.
- Only labelled products are judged: unlabelled products are skipped in the ranking, and relevant products the AND search misses count as missed.
- For our queries, R is the number of relevant products in the judged top 20, so recall is relative to that pool.
- Our labels follow the criteria of Table 6, with the strict option for "running" shoes.

11 AI usage

We used Claude Sonnet 5.5 as a support tool to review the code and suggest coding best practices, such as refactoring some of our code. We reviewed all its suggestions, ran the code ourselves, checked every number against the notebook and edited the final text. The relevance judgements and criteria are also our own judgements on the queries. We learnt LaTeX for this report, and Sonnet helped us remember some of its command names. We also used Sonnet to check for grammar and spelling mistakes.